CANOPIE-HPC Workshop

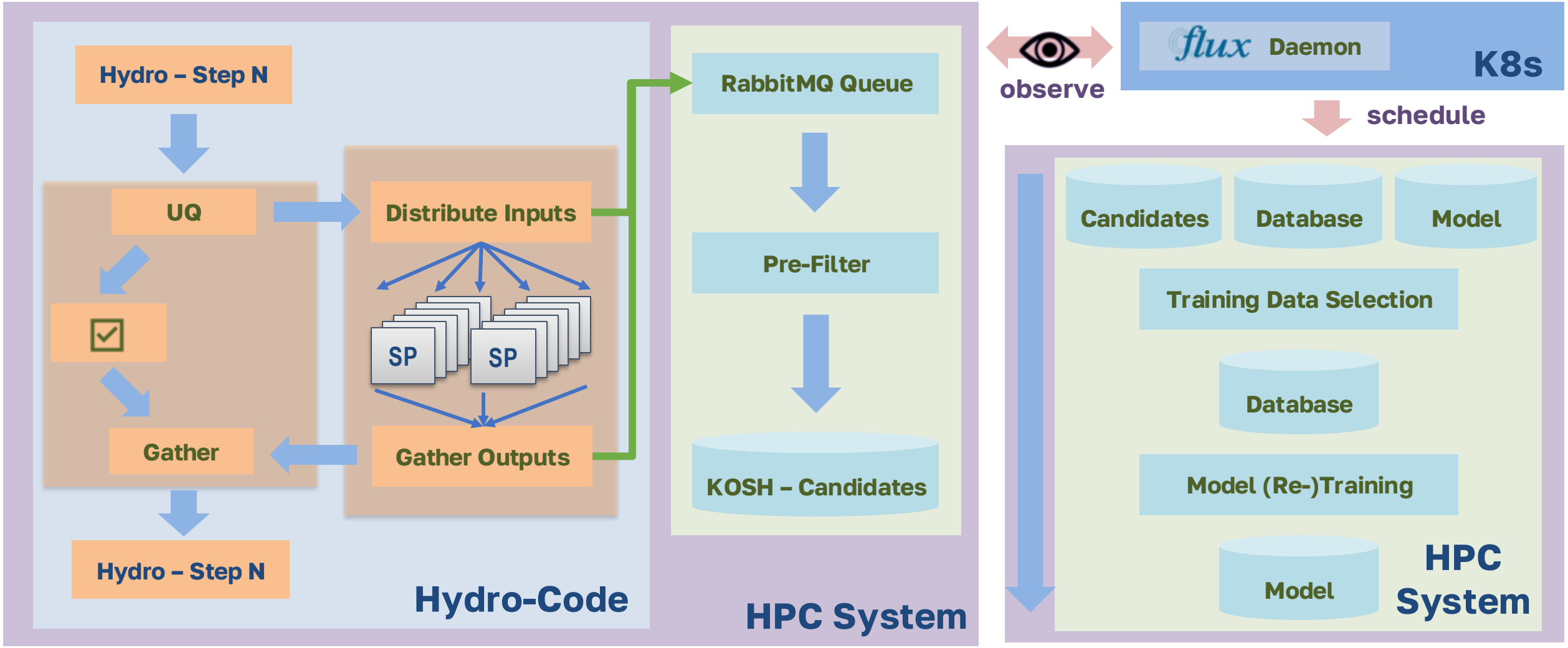
Usability Evaluation of Cloud for HPC Applications

Vanessa Sochat, **Daniel Milroy**, Abhik Sarkar, Aniruddha Marathe, and Tapasya Patki
Lawrence Livermore National Laboratory

SC25 hpc ignites.
ST. LOUIS

LLNL-PRES-2013239 This work was performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under Contract DE-AC52-07NA27344.

Complex workflows integrate MPI-based simulation with ensemble methods, services, communications, and AI/ML.



https://github.com/LLNL/AMS

Converged computing is "the best of both worlds:" HPC+Cloud

The cloud is an environment (public, private) that supports:

- Portability, reproducibility (e.g., containerization)
- Resiliency, efficiency (e.g., resource dynamism, elasticity, declarative management)
- Reduced complexity via automation (autoscaling, elasticity, declarative management)

Companies rent this environment; profitable, dominating software and hardware innovation space

- projected to $1.28T USD revenue by 2028, 20% CAGR[1] vs HPC: $100B by 2028, CAGR 8%[2]

Current funding landscape is shaky for science

Portability is a new dimension of performance

[1]Gartner 2024, [2]Hyperion 2025,

Purpose and contributions of this work

Exploration, testing new environments (K8s vs VMs) vs bare metal. Building software to enable others to do the same.

- Suggested practices for orchestration of cloud studies
- Usability assessment of 12 cloud environments (1 failed)
- Scaling tests to 256 nodes, GPUs across 13 environments
- Kubernetes configurations with 11 HPC apps on 32-256 nodes
- Daemonset to deploy InfiniBand on Azure
- 97 containers for CPU/GPU HPC apps
- Azure VM bases for Ubuntu 24.04 with NVIDIA drivers
- 3,546 runs reported in the paper out of 25,541
- Presentations to and feedback from all three clouds!

Data, configurations, scripts, etc.: https://doi.org/10.5281/zenodo.13738495

If we need apps to be portable, which scenario makes sense?

Optimize perfectly for one environment.

Build well for many environments

Do we understand the environments?

We assessed cloud utility for HPC applications

Usability

- How easy (or hard) was it to use X?
- Were we able to get resources?
- Did we experience unexpected errors?
- What is environment X suitable for?
- How easy (or hard) was it to prototype?

Performance

- Did application strong/weak scale?
- How did network perform?
- How consistently did applications run?

The first challenge was choosing the applications and test configurations based on the budget.

Dependency between:

- Application choice and quantity
- Container base
- Build configurations: confidence apps can be built on untested platforms
- Run configurations: confidence they can run? Fit in memory on possible instances and on-prem? Representative test? Not too long or too short?
- Projected maximum node count?

And the budget of $150,000…

The first challenge was choosing the applications and test configurations based on the budget.

We prototyped with on-premises systems, clear-eyed of the risks.

- Built dozens of applications, selecting most likely to port within time/budget constraints
- Tested execution parameters with scaling studies on-premises, introducing correction factors for estimated scaling on cloud (risky due to untested platforms)
- Couldn’t test all cloud environments due to initial contractual challenges

Process took > 4 months, exposing budget timeline and restriction risks

We tested 12 applications for GPU and CPU

Applications

GPU and CPU		GPU only
AMG2023	Mixbench	Magma
Kripke	OSU Benchmarks	Multi GPU models
Laghos	Quicksilver	
Lammps	Single node metrics	
Minife	Stream	

32-256 CPU nodes; 32-256 GPUs (4-64 nodes)

Deployed across clouds and environments

Microsoft Azure

- CycleCloud CPU (32-256 nodes)
- CycleCloud GPU (4-32 nodes)
- AKS CPU (32-256 nodes)
- AKS GPU (4-32 nodes)

Google Cloud

- Compute Engine CPU (32-256 nodes)
- Compute Engine GPU (4-32 nodes)
- GKE CPU (32-256 nodes)
- GKE GPU (4-32 nodes)

Amazon Web Services (AWS)

- ParallelCluster CPU (32-256 nodes)
- EKS CPU (32-256 nodes)
- EKS GPU (4-32 nodes)

Applications

GPU and CPU

AMG2023
Kripke
Laghos
Lammps
Minife
Mixbench
OSU Benchmarks
Quicksilver
Single node metrics
Stream

GPU only

Magma
Multi gpu models

Deployed across clouds and environments

Microsoft Azure

- CycleCloud CPU (32-256 nodes)
- CycleCloud GPU (4-32 nodes)
- AKS CPU (32-256 nodes)
- AKS GPU (4-32 nodes)

Google Cloud

- Compute Engine CPU (32-256 nodes)
- Compute Engine GPU (4-32 nodes)
- GKE CPU (32-256 nodes)
- GKE GPU (4-32 nodes)

Amazon Web Services (AWS)

- ParallelCluster CPU (32-256 nodes)
- EKS CPU (32-256 nodes)
- EKS GPU (4-32 nodes)

Applications

GPU and CPU

AMG2023
Kripke
Laghos
Lammps
Minife
Mixbench
OSU Benchmarks
Quicksilver
Single node metrics
Stream

GPU only

Magma
Multi gpu models

Deployed across clouds and environments

Microsoft Azure

- CycleCloud CPU (32-256 nodes)
- CycleCloud GPU (4-32 nodes)
- AKS CPU (32-256 nodes)
- AKS GPU (4-32 nodes)

Google Cloud

- Compute Engine CPU (32-256 nodes)
- Compute Engine GPU (4-32 nodes)
- GKE CPU (32-256 nodes)
- GKE GPU (4-32 nodes)

Amazon Web Services (AWS)

- ParallelCluster CPU (32-256 nodes)
- EKS CPU (32-256 nodes)
- EKS GPU (4-32 nodes)

Applications

GPU and CPU

AMG2023	Mixbench
Kripke	OSU Benchmarks
Laghos	Quicksilver
Lammps	Single node metrics
Minife	Stream

GPU only

Magma
Multi gpu models

Node and Network Details

Table 2: Nodes and Network

Environment	Node Type	Processor/GPU	Cores/Frequency	Memory	Network	Cost/Hr
CPU *A* (p)	Dell	Intel Xeon Platinum 8480+	112/3.8 GHz	256GB	Omni-Path 100	–
CPU ParallelCluster (aws)	Hpc6a	AMD EPYC 7R13/7003	96/3.6 GHz	384GB	EFA Gen1.5	$2.88
CPU EKS (k,aws)	Hpc6a	AMD EPYC 7R13/7003	96/3.6 GHz	384GB	EFA Gen1.5	$2.88
CPU Compute Engine (g)	c2d-standard-112	AMD EPYC 7B13	56/2.45-3.5 GHz	448GB	Google Premium	$5.06
CPU GKE (k,g)	c2d-standard-112	AMD EPYC 7B13	56/2.45-3.5 GHz	448GB	Google Premium, Tier_1	$5.06
CPU CycleCloud (az)	HB96rs v3	AMD EPYC 7003	96/1.9-3.5 GHz	448GB	InfiniBand HDR	$3.60
CPU AKS (k,az)	HB96rs v3	AMD EPYC 7003	96/1.9-3.5 GHz	448GB	InfiniBand HDR	$3.60
GPU *B* (p)	IBM	IBM Power9/V100 16GB	44/3.5 GHz max	256GB	InfiniBand EDR	–
GPU ParallelCluster (aws)	p3dn.24xlarge	Xeon Platinum 8175/V100 32GB	48/2.5 GHz	768GB	EFA Gen1	$34.33
GPU EKS (k,aws)	p3dn.24xlarge	Xeon Platinum 8175/V100 32GB	48/2.5 GHz	768GB	EFA Gen1	$34.33
GPU Compute Engine (g)	n1-standard-32	Xeon Haswell E5 v3/V100 16GB	16/2.3 GHz	120GB	Google Premium	$23.36
GPU GKE (k,g)	n1-standard-32	Xeon Haswell E5 v3/V100 16GB	16/2.3 GHz	128GB	Google Premium	$23.36
GPU Cyclecloud (az)	ND40rs v2	Xeon Platinum 8168/V100 32GB	48/2.7 GHz	672GB	InfiniBand EDR	$22.03
GPU AKS (k,az)	ND40rs v2	Xeon Platinum 8168/V100 32GB	48/2.7 GHz	672GB	InfiniBand EDR	$22.03

Amazon Web Services *aws*, Microsoft Azure *az*, Google Cloud *g*, On-Premises *p*, Kubernetes *k*. Cost is hourly in USD for a single instance, with GPUs included.

We chose resources based on availability for the sizes we are interested in testing, and commonality across clouds.

Most “fair” subject to availability at scale was the only choice

Some of the applications were originally developed for NVIDIA V100

Goal was to test at moderate HPC scale, comparing to on-premises clusters.

(available in quantity across clouds)

(not available in quantity across clouds) *"best"*

older generation

newest generation

High-level study design

For each environment (CPU/GPU)

- Bring up the cluster (cloud only)
- Setup or install dependencies
- For each application:
  - Run 5 iterations
  - < 15 mins / iteration, 30 mins total
  - Save logs, metrics, output
- Bring down cluster
- Record / reflect on usability experience

Usability Results

Environment Usability - Assessment of Effort

Environment		Accelerator?	Setup	Development	Application Setup	Manual Intervention
Amazon Web Services	ParallelCluster	CPU	medium	low	low	low
Microsoft Azure	CycleCloud	CPU	high	low	high	high
Google Cloud	Compute Engine	CPU	medium	medium	low	low
Microsoft Azure	CycleCloud	GPU	high	low	high	high
Google Cloud	Compute Engine	GPU	medium	medium	low	low
Amazon Web Services	Kubernetes	CPU	low	high	low	medium
Microsoft Azure	Kubernetes	CPU	medium	high	high	high
Google Cloud	Kubernetes	CPU	low	low	low	medium
Amazon Web Services	Kubernetes	GPU	high	high	low	medium
Microsoft Azure	Kubernetes	GPU	medium	high	high	medium
Google Cloud	Kubernetes	GPU	low	low	low	medium
Institutional	On-premises	GPU	low	low	high	medium
Institutional	On-premises	CPU	low	low	high	medium

- **low**: procedure required for the step worked after following available instructions, with minimal configuration or development on our part.
- **medium**: facing unexpected issues to be resolved by debugging or development
- **high**: significant effort needed for components of the environment to function

AWS: ParallelCluster (Slurm)

Environment		Accelerator?	Setup	Development	Application Setup	Manual Intervention
Amazon Web Services	ParallelCluster	CPU	medium	low	low	low

- **Setup (CPU)**:
  - Configuration files complex - one file to create subnets/security groups, copy paste into a second
  - Once deployed, user experience was good.
- **Setup (GPU) couldn't complete**:
  - required a custom image build and multi-step configuration.
  - We didn't get GPU image working in reservation time (48 hours for all experiments)

- **low**: procedure required for the step worked after following available instructions, with minimal configuration or development on our part.
- **medium**: facing unexpected issues to be resolved by debugging or development
- **high**: significant effort needed for components of the environment to function

Microsoft Azure: CycleCloud (Slurm)

Environment		Accelerator?	Setup	Development	Application Setup	Manual Intervention
Microsoft Azure	CycleCloud	CPU	high	low	high	high
Microsoft Azure	CycleCloud	GPU	high	low	high	high

- Excellent support – if we had an issue? Email → "Let's jump on a call" often within the hour.
- **Setup**: Each of CPU and GPU setups took over a day to deploy and interfaces were challenging to use
  - Often went out of sync with the Azure portal
  - Changing state in the cluster didn't change the interface and vice versa
- **Application Setup**: Azure did not provide container bases – we developed custom bases
  - No suggested best practices for using UCX - we found different optimal values depending on environment
- **Manual Intervention**: Constant monitoring needing to ensure jobs were not stalled
  - Issues with process management, module loading, Slurm, the environment.

- **low**: procedure required for the step worked after following available instructions, with minimal configuration or development on our part.
- **medium**: facing unexpected issues to be resolved by debugging or development
- **high**: significant effort needed for components of the environment to function

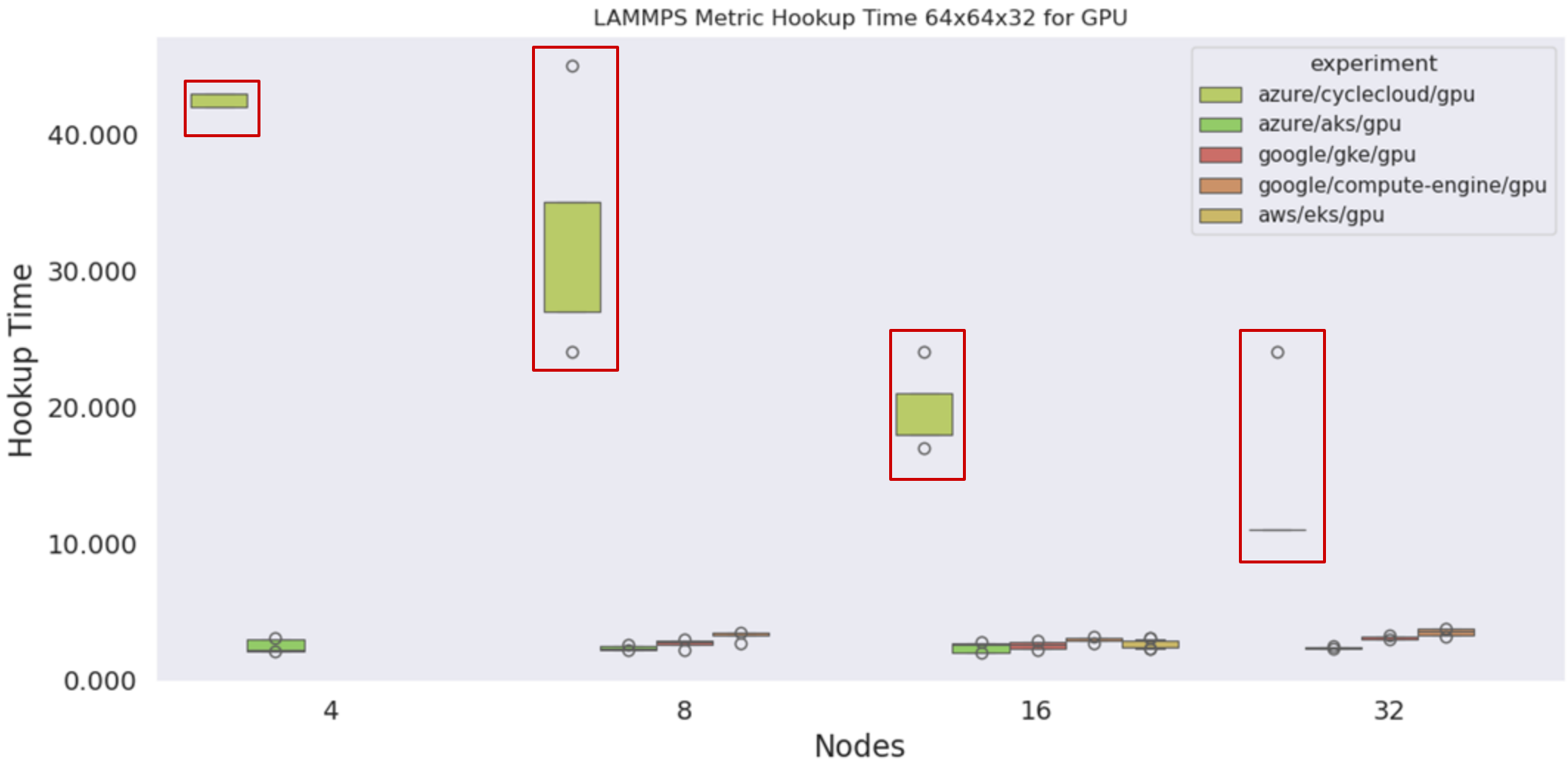
Azure Environments have "Hookup Time"

Subtract from the wrapping time reported by the workload manager

```
Loop time of 29.8989 on 768 procs for 100 steps with 1245184 atoms

Performance: 0.029 ns/day, 830.526 hours/ns, 3.345 timesteps/s, 4.165 Matom-step/s
99.2% CPU use with 768 MPI tasks x 1 OpenMP threads

MPI task timing breakdown:
Section |  min time  |  avg time  |  max time  |%varavg| %total
---------------------------------------------------------------
Pair    | 17.164     | 20.227     | 23.037     |  21.9 | 67.65
Neigh   | 0.20885    | 0.21279    | 0.2188     |   0.6 |  0.71
Comm    | 0.09463    | 2.8066     | 5.9358     |  59.0 |  9.39
Output  | 0.0038702  | 0.08082    | 0.20756    |  23.7 |  0.27
Modify  | 6.5366     | 6.5688     | 6.5966     |   0.5 | 21.97
Other   |            | 0.002413   |            |       |  0.01

Nlocal:        1621.33 ave        1670 max        1589 min
Histogram: 151 197 158 6 0 0 0 2 148 106
Nghost:        8636.47 ave        8804 max        8529 min
Histogram: 222 282 8 0 0 0 0 0 12 244
Neighs:         562779 ave      577262 max      552472 min
Histogram: 122 173 182 35 0 0 0 1 146 109

Total # of neighbors = 4.3221461e+08
Ave neighs/atom = 347.10903
Neighbor list builds = 5
Dangerous builds not checked
Total wall time: 0:00:30
```

Azure Environments have "Hookup Time"

We still do not understand the cause



ECC not enabled uniformly across GPUs

Azure is the only cloud with a "mixture"

- Highly undesirable; unknown consequences for codes

9	google/gke/gpu	gpu	32	ecc_enabled	Yes	1.0
27	google/gke/gpu	gpu	8	ecc_enabled	Yes	1.0
45	google/gke/gpu	gpu	16	ecc_enabled	Yes	1.0
63	google/gke/gpu	gpu	4	ecc_enabled	Yes	1.0
81	google/compute-engine/gpu	gpu	32	ecc_enabled	Yes	1.0
99	google/compute-engine/gpu	gpu	8	ecc_enabled	Yes	1.0
117	google/compute-engine/gpu	gpu	16	ecc_enabled	Yes	1.0
135	google/compute-engine/gpu	gpu	4	ecc_enabled	Yes	1.0
153	aws/eks/gpu	gpu	8	ecc_enabled	Yes	1.0
171	aws/eks/gpu	gpu	16	ecc_enabled	Yes	1.0
189	aws/eks/gpu	gpu	4	ecc_enabled	Yes	1.0
207	azure/cyclecloud/gpu	gpu	32	ecc_enabled	Yes	0.75
208	azure/cyclecloud/gpu	gpu	32	ecc_enabled	No	0.25
226	azure/cyclecloud/gpu	gpu	8	ecc_enabled	No	0.425
227	azure/cyclecloud/gpu	gpu	8	ecc_enabled	Yes	0.575
245	azure/cyclecloud/gpu	gpu	16	ecc_enabled	Yes	0.625
246	azure/cyclecloud/gpu	gpu	16	ecc_enabled	No	0.375
264	azure/cyclecloud/gpu	gpu	4	ecc_enabled	Yes	0.5
265	azure/cyclecloud/gpu	gpu	4	ecc_enabled	No	0.5
283	azure/aks/gpu	gpu	32	ecc_enabled	Yes	0.71875
284	azure/aks/gpu	gpu	32	ecc_enabled	No	0.28125
302	azure/aks/gpu	gpu	8	ecc_enabled	Yes	0.875
303	azure/aks/gpu	gpu	8	ecc_enabled	No	0.125
321	azure/aks/gpu	gpu	16	ecc_enabled	Yes	0.6875
322	azure/aks/gpu	gpu	16	ecc_enabled	No	0.3125
340	azure/aks/gpu	gpu	4	ecc_enabled	Yes	1.0

We brought the hookup time and ECC issues up with Azure, offered to help debug, and we didn’t hear back.

Google Cloud: Compute Engine (Flux)

Environment		Accelerator?	Setup	Development	Application Setup	Manual Intervention
Google Cloud	Compute Engine	CPU	medium	medium	low	low
Google Cloud	Compute Engine	GPU	medium	medium	low	low

- **Setup**: Had trouble with Cluster Toolkit setup with Slurm
  - We could ultimately not get Cluster Toolkit deployed with NVIDIA V100 and Slurm
- **Development**: Needed to develop Terraform deployment with Flux to run study

- **low**: procedure required for the step worked after following available instructions, with minimal configuration or development on our part.
- **medium**: facing unexpected issues to be resolved by debugging or development
- **high**: significant effort needed for components of the environment to function

Insight: when possible, explainability is expensive

There is an assumption in the HPC community we can deeply probe a system, and that we don't have to pay for the investigation.

- Trade-off between time spent profiling and optimizing and running the application for scientific output
  - Clouds provision only a small set of instance types with performance counters available.
  - Examine the amount of time to identify and fix performance bottlenecks vs. simply running the application

Kubernetes!

AWS EKS Kubernetes: (Flux Operator)

Environment		Accelerator?	Setup	Development	Application Setup	Manual Intervention
Amazon Web Services	Kubernetes	CPU	low	high	low	medium
Amazon Web Services	Kubernetes	GPU	high	high	low	medium

- **Setup**: GPU acquisition using eksctl erroneously created a placement group, led to partial instantiation of cluster
  - Required debugging, significant time/cost and custom build (and pull request) to repository to fix.
  - We wound up getting GPU nodes that were in different availability zones!
- **Development**: Missing cleanup of EFA installer broke portion of nodes on provisioning (custom build required)
  - Ran out of network prefixes at largest cluster size for CNI, required custom patch of daemonset
    - Needed to allow for prefix delegation to increase addresses available
- **Manual Intervention**: After manual creation, needed to shell in to interact with queue.

- **low**: procedure required for the step worked after following available instructions, with minimal configuration or development on our part.
- **medium**: facing unexpected issues to be resolved by debugging or development
- **high**: significant effort needed for components of the environment to function

Azure AKS Kubernetes: (Flux Operator)

Environment		Accelerator?	Setup	Development	Application Setup	Manual Intervention
Microsoft Azure	Kubernetes	CPU	medium	high	high	high
Microsoft Azure	Kubernetes	GPU	medium	high	high	medium

- **Setup**: Multiple setup steps involving command line and interface
  - For GPU, instances would come up with only 7/8 GPUs
- **Development**: Required custom development of Infiniband driver installer
- **Application Setup**:
  - Same issue with UCX parameters (but different this time)
  - Needed to create custom containers to support specialized software (e.g., HPCX, HCOLL, SHARP)
- **Manual Intervention**: Operations asking for proximity placement groups would not complete for > 100 nodes

- **low**: procedure required for the step worked after following available instructions, with minimal configuration or development on our part.
- **medium**: facing unexpected issues to be resolved by debugging or development
- **high**: significant effort needed for components of the environment to function

Google Cloud GKE Kubernetes: (Flux Operator)

Environment		Accelerator?	Setup	Development	Application Setup	Manual Intervention
Google Cloud	Kubernetes	CPU	low	low	low	medium
Google Cloud	Kubernetes	GPU	low	low	low	medium

- **Manual Intervention**: After manual creation, needed to shell in to interact with queue.
- Notable differences in GPU provisioning between paid and credit-based account.
  - Likely a result of different levels of account reputation

- **low**: procedure required for the step worked after following available instructions, with minimal configuration or development on our part.
- **medium**: facing unexpected issues to be resolved by debugging or development
- **high**: significant effort needed for components of the environment to function

On-Premises:

Environment		Accelerator?	Setup	Development	Application Setup	Manual Intervention
Institutional	On-premises	GPU	low	low	high	medium
Institutional	On-premises	GPU	low	low	high	medium

- **Application Setup (controversial?)**: Largely no control over software environment (environment modules)
- **Manual Intervention**: Jobs would error and had to be monitored and debugged.
  - Not uncommon to get bad nodes
  - Significant time waiting in the queue (not paying)

Setup and Development "low" scores reflect having an entire admin team to manage the environment!

- **low**: procedure required for the step worked after following available instructions, with minimal configuration or development on our part.
- **medium**: facing unexpected issues to be resolved by debugging or development
- **high**: significant effort needed for components of the environment to function

Insight: Like in HPC-native, problems emerge with scale

- Larger scale more likely to expose bugs
- Burden is on the user to debug and figure out
- Cloud vendors are generally responsive to feedback

Insight: Early adoption means wearing many hats

What are differences between using cloud and HPC?

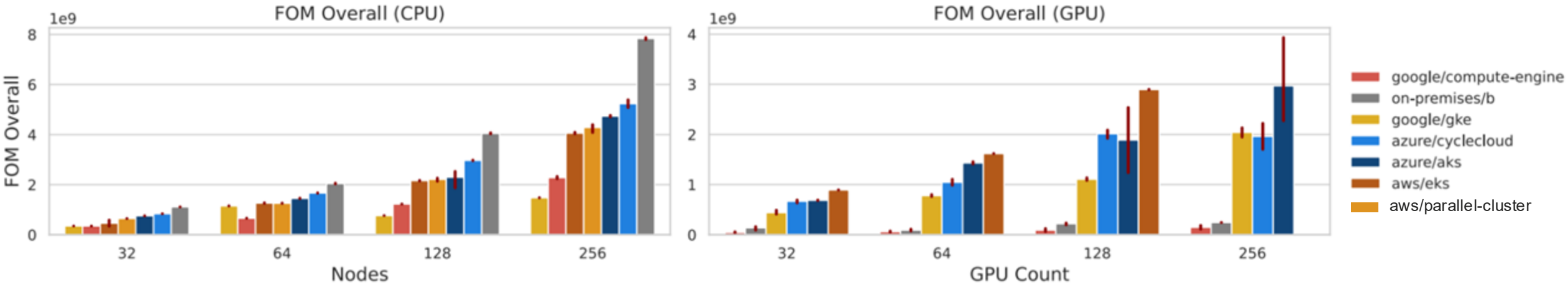
	HPC	Cloud	Roles
Planning	Allocation / cluster available to you "free"	You plan and monitor all costs	project manager
Resources	Pre-existing and setup for you	You choose, you manage: cost estimates, setup, deploy, teardown	system administrator
Software	Driver software pre-installed and loadable Build "from scratch" with spack, etc.	You build it (containers).	software developer
Infrastructure	Provided by vendors or production installs	Something missing? You develop it.	cloud engineer
Debugging	Submit a ticket to support staff	You are the support staff	dev-ops engineer

If your team is missing any of these roles, that is your stopping point for using cloud.

Performance Results

AMG

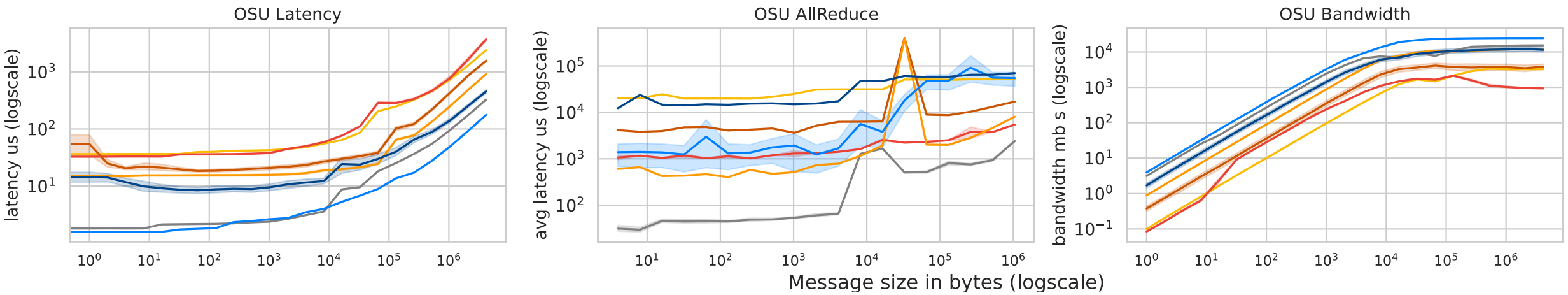
Algebraic Multi-grid Solver (higher FOM is better)



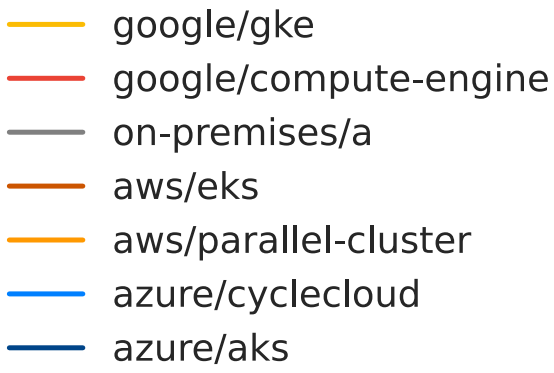
- Topology varies based on tasks. In testing, processor topology had <10% influence on FOM
- Significant issues with building container
- GPU problem size chosen to fit in V100 memory on Google Cloud and Lassen (16GB)
- Cloud GPU environments excelled, on-premises runs had highest FOMs for CPU
- Azure CPU environments lead, AWS EKS leads GPU until max cluster size
- ParallelCluster higher FOM than EKS (CPU), AKS leads CCloud (GPU), CCloud leads AKS (CPU), Google…

OSU Benchmarks

Size 256 Cluster for cloud and on-premises environments

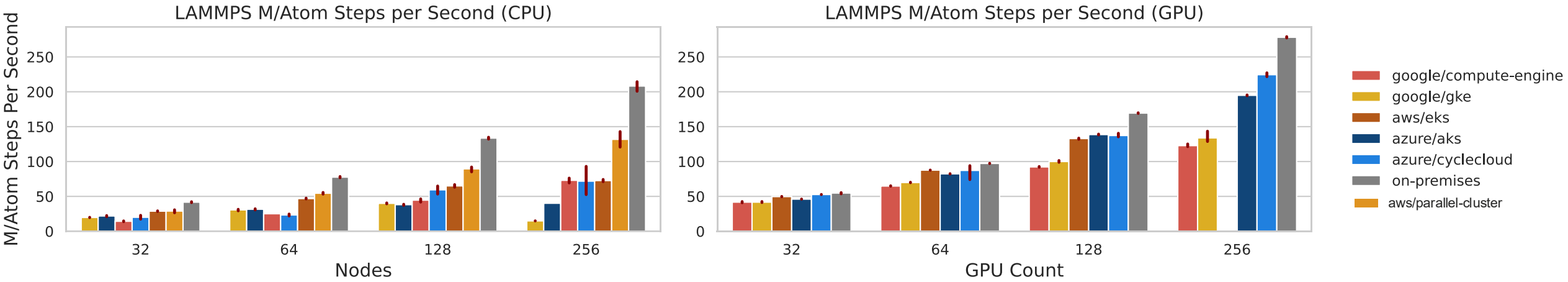


- InfiniBand and on-premises had lowest p2pt latencies, highest bandwidth
- CycleCloud had highest variation within runs for AllReduce
- AWS ParallelCluster had second lowest latency for AllReduce; Google CE third



LAMMPS

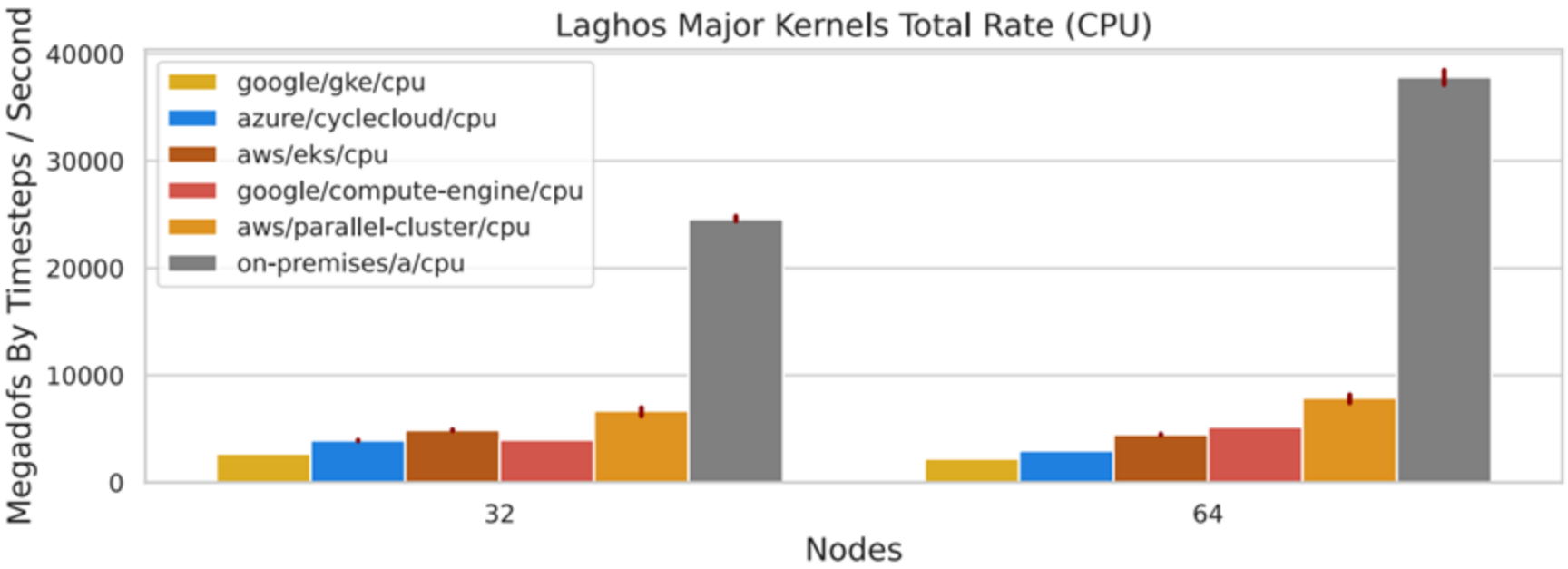
Large-scale Atomic/Molecular Massively Parallel Simulator



- CPU problem size 64 x 64 x 32; GPU: 64 x 32 x 32
- GPU runs not possible on AWS ParallelCluster or largest size of AWS EKS
- AWS fastest cloud for CPU, Azure for GPU
- On-premises uniformly fastest

Laghos

FOM is Major kernels total rate (megadofs x time steps / second)

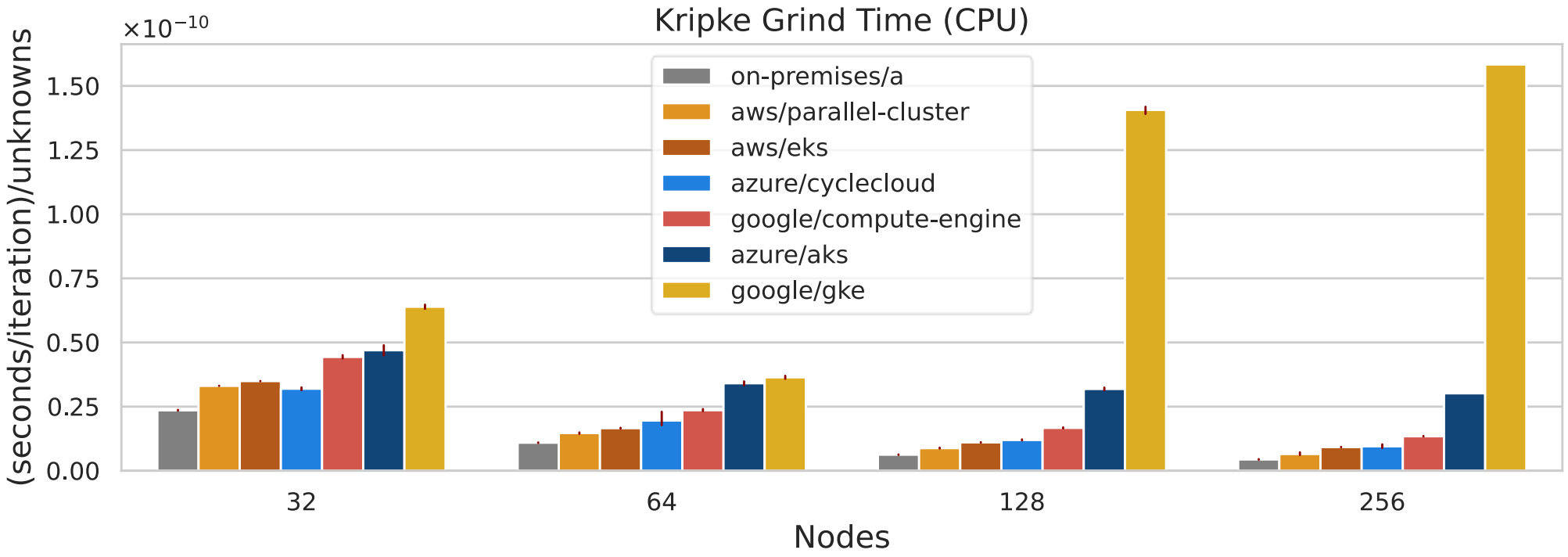


- Larger values are better
- Infeasible to run on cloud environments. Didn't complete beyond 64 nodes in under 20 minutes

More optimization is needed since it doesn't scale!

Kripke

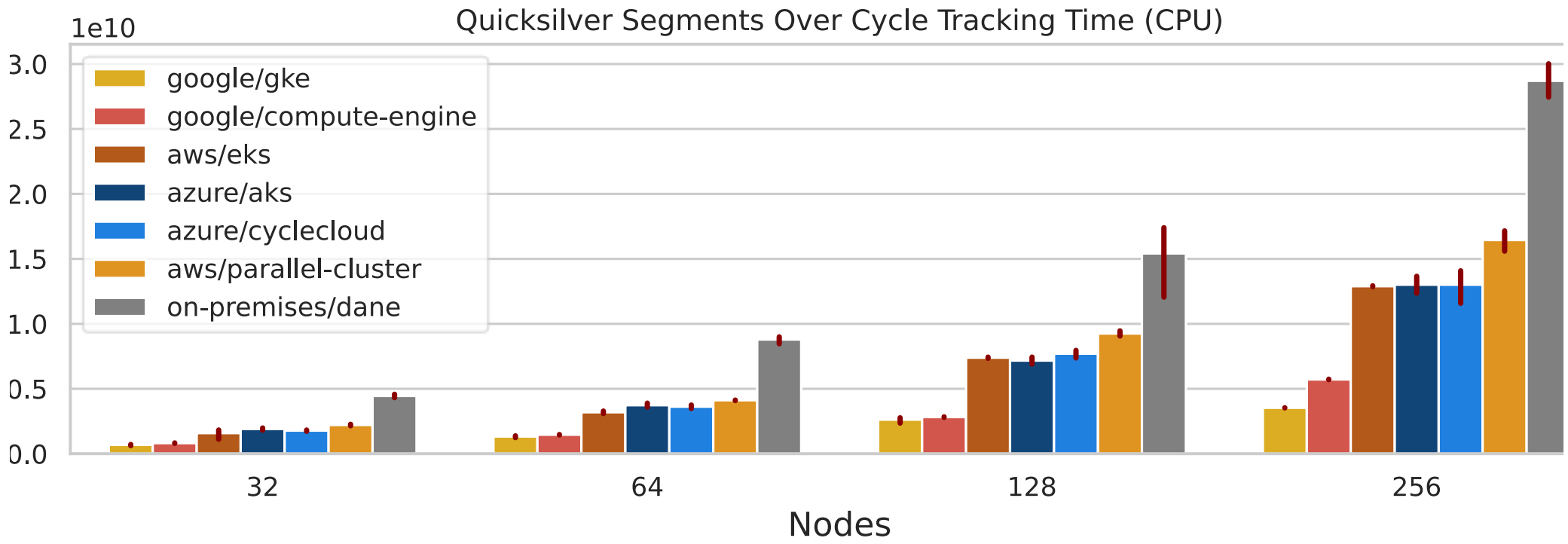
deterministic particle transport proxy application intended for testing how data, programming, and systems impact particle transport performance



- FOM: grind time; lower is better
- AWS leads after on-premises
- EKS leads ParallelCluster, AKS leads Cyclecloud
- Difficulty binding with GPUs; results not included
- Infeasible to run on GKE: follow-up study to understand why

Quicksilver

is a proxy application that solves a simplified Monte Carlo particle transport problem



- FOM: segments over cycle tracking time; larger values better
- AWS bookends Azure, on-premises dominates
- K8s and VMs exhibit similar performance until 128 nodes

Performance Results Summary

Can we classify application behaviors in these environments?

Cloud readiness highlights GPU:

- Cloud platforms can outperform on-premises systems: see AMG2023 GPU
- LAMMPS exhibits comparable performance to on-premises, especially with GPU

Application-specific sensitivity

- Methods are needed to determine sources of overhead and performance bottlenecks (e.g., eBPF, profiling, tracing)

Communication (latency) bound

- Applications that show zero scaling need more work, either on the build or overall design.
- OSU allreduce could be a hint

We've raised more questions than we've answered…

Kubernetes facilitates orchestration

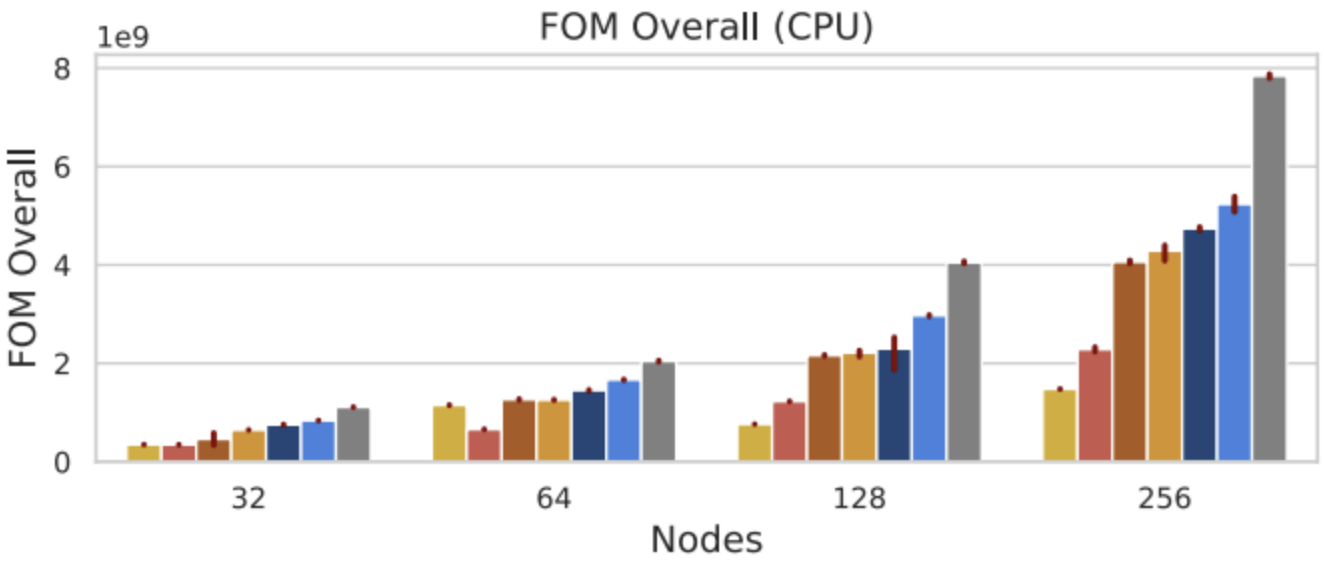
And we want it to perform on-par with VM environments

- Performance disparity between VMs and Kubernetes
  - Kubernetes offers better automation, orchestration, overall usability, sometimes better performance
    - Sometimes VM-K8s performance delta changes sign (e.g., 64 to 128, below)
    - We want to understand and resolve this discrepancy.

Overall FOM for AMG2023

Google Compute Engine

Google Kubernetes Engine (GKE)



We need to be realistic about resource contention

HPC is not driving innovation

- Lower precision chips are being developed for predominant AI/ML use cases
- We must respond to the change and best use resources that are available to us.
  - Older GPUs might be easier to get quota for / to access

A possible future is scientific work running where it was not intended

Final thoughts

Diverse R&D Teams

Members run the gamut from software developer to system administrator to dev-ops or cloud engineer.

When you get something working, share with the larger community:

- Our Infiniband Daemonset Installer for Azure **[1]**
- Azure VM Bases with Ubuntu 24.04 / open source stack **[2]**

[1] https://github.com/converged-computing/aks-infiniband-install

[2] https://github.com/converged-computing/flux-tutorials/tree/main/tutorial/azure

This study was over a year ago, and *the landscape has changed!*

- Earlier in the year we tried to reproduce setup on AWS to do reruns of OSU…
  - We spent upwards of $2,000 trying (unsuccessfully) to get size 256 cluster on AWS
- Attempting the same for Azure…
  - setup did not reproduce - MPI experienced errors
- AWS has dropped GPU prices for specific families by 25-45% to make more accessible
- Google released their first "GKE and GPUDirect RDMA with DRA"

Adapting to change is required. We've observed inconsistency working in these environments across a few months.

Thank you!

Questions?

sochat1@llnl.gov | milroy1@llnl.gov

Lawrence Livermore National Laboratory